

← Go Back

Hardware

GIGA R1 WiFi

Tutorials

- Arduino GIGA R1 User Manual
- Guide to GIGA R1 Advanced ADC/DAC and Audio Features
- GIGA R1 Camera Guide
- Dual Core Debugging the GIGA R1 WiFi
- Guide to GIGA R1 Dual Cores
- Getting Started with GIGA R1 WiFi
- MicroPython on the GIGA R1
- Guide to Arduino GIGA USB Features**
- GIGA R1 WiFi Network Examples

Home / Hardware / GIGA R1 WiFi / **Guide to Arduino GIGA USB Features**

Guide to Arduino GIGA USB Features

Learn how you can turn your USB device into a mouse or keyboard, how to read & write to a USB mass storage, and connecting a keyboard via the USB-A connector.

Author · Karl Söderby · Last revision · 12/16/2024

ON THIS PAGE

Hardware & Software Needed

- Libraries
- USB Overview
- Enable/Disable USB-A Port
- USB Mass Storage +
- USB Host Keyboard
- USB HID +
- Summary

The GIGA R1 comes with a great range of supported USB features, including **USB HID**, **USB Host** and **Mass Storage**. It also comes with a USB-A connector that allow you to plug devices directly to your GIGA R1 board.

It can easily be configured to act as a mouse or keyboard (HID device) or as a USB Host (connect a keyboard, USB stick). This makes it possible to create powerful interfaces, and build complex interactive projects.

In this guide, we will take a look at the available features, how to enable them in a sketch and what circuit (if any) is required.

Hardware & Software Needed

Arduino GIGA R1 WiFi.

USB Mass Storage Device (USB Stick).

Keyboard.

Libraries

USB features on the GIGA R1 is currently enabled through three separate libraries.

USBHID - included in the Board Package.

Arduino_USBHostMbed5 - can be downloaded through the Arduino IDE.

USBHostGiga (Alpha) - library only available through its GitHub

These libraries each provides a set of methods to access the USB features on the board. This includes using the GIGA R1 as a mouse/keyboard (HID), reading & writing to mass storage devices & connecting external keyboards.

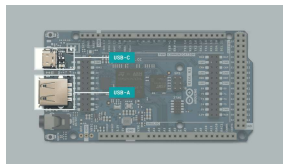
The examples provided in this guide are based on these libraries.

USB Overview

The GIGA R1 has two USB connectors:

USB-C - for powering, programming & HID communication.

USB-A - for USB-Host (connecting keyboards, USB sticks etc).

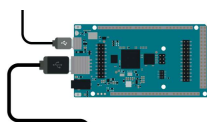


USB ports on GIGA R1.

Note: do NOT connect the USB-A connector to your computer. This is not a programming port and if the **PA15** pin is enabled, it can damage your computer's USB port.



Below is an example of how you should use the USB-C / USB-A connectors.



Keyboard connected to
USB-A port.

Enable/Disable USB-A Port

To enable the USB-A port, you will need to define it as an output. You can also disable it inside of a sketch by writing a LOW state to it.

```
1 void setup(){  
2   pinMode(PA_15, OUTPUT)  
3 }  
4  
5 void loop(){  
6   digitalWrite(PA_15, H  
7   digitalWrite(PA_15, L  
8 }
```

Please note that connecting
devices to the USB-A that



draw high amounts of
current may cause the
board to reset.

USB Mass Storage

The USB-A connector onboard the GIGA R1 can be used to connect **USB mass storage devices**, for example, a USB stick. This can be used for a number of applications, including:

Accessing large files such as
images & audio files,

Logging large amounts of
data,

USB mass storage devices connected needs to be formatted with the **FAT32** as a file system, using the **MBR partitioning scheme**. This is a requirement, and reading & writing will not work otherwise.

The USB mass storage features are based on the [Arduino_USBHostMbed5](#) library.

USB Designation

To access the correct USB mass storage device, we need to specify the **designation** in the code.

```
1 mbed::FATFileSystem usb
```

This is so that our GIGA R1 can target the right USB device. In addition, when reading & writing to files, please ensure that the designation is included in the path.

The below line of code is an example of how you need to use the designation in the path, `/usb/` before accessing the file.

```
1 FILE *f = fopen("/usb/t
```



The designation is essential to consider when using the examples in this guide. All examples are based on the designation = `usb`. You will need to change this in the code, or else it will not work.

List File Directory

Below is an example sketch that can be used to **list** files in a USB mass storage device.

```
1  #include <DigitalOut.h>
2  #include <FATFileSyst
3  #include <Arduino_USB
4
5  USBHostMSD msd;
6  mbed::FATFileSystem u
7
8
9  void setup()
10 {
11     Serial.begin(1152
12
13     pinMode(PA_15, OU
14     digitalWrite(PA_1
15
16     while (!Serial)
17         ;
18
19     Serial.println("S
20
21     // if you are usi
22     // start_hub();
23
24     while (!msd.conne
25         //while (!por
26         delay(1000);
27 }
28
```

File Read

Below is an example sketch that can be used to **read** files from a USB mass storage device.

```
1  #include <Arduino_USB
2  #include <DigitalOut.
3  #include <FATFileSyst
4
5  USBHostMSD msd;
6  mbed::FATFileSystem u
7
8  // If you are using a
9  mbed::DigitalOut otg(
10
11 void setup() {
12     Serial.begin(115200
13
14     pinMode(PA_15, OUTP
15     digitalWrite(PA_15,
16
17     while (!Serial);
18
19     delay(2500);
20     Serial.println("Sta
21
22     // if you are using
23     //start_hub();
24
25     while (!msd.connect
26         delay(1000);
27     }
28
```

File Write

Below is an example sketch that can be used to **write** files from a USB mass storage device.

```

1  #include <Arduino_USB
2  #include <DigitalOut.
3  #include <FATFileSyst
4
5  USBHostMSD msd;
6  mbed::FATFileSystem u
7
8  // mbed::DigitalOut p
9  mbed::DigitalOut otg(
10
11 void setup() {
12     Serial.begin(115200
13
14     pinMode(PA_15, OUP
15     digitalWrite(PA_15,
16
17     while (!Serial);
18
19     msd.connect();
20
21     while (!msd.connect
22         //while (!port.co
23         delay(1000);
24     }
25
26     Serial.println("Mou
27     int err = usb.moun
28     if (err) {

```

Datalogger Example

In the example below, we are reading logging the `A0` pin, where we are defining two parameters:

`interval` - how long between each reading.

`number_of_readings` - how many readings we should take.

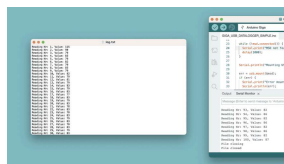
This is useful if you e.g. want to log a specific amount of samples for a specific amount of time.

```

1  #include <Arduino_USB
2  #include <DigitalOut.
3  #include <FATFileSyst
4
5  USBHostMSD msd;
6  mbed::FATFileSystem u
7
8  int err;
9  int count;
10 int number_of_reading
11 int interval = 10; //
12
13 void setup() {
14     Serial.begin(115200
15
16     pinMode(PA_15, OUTP
17     digitalWrite(PA_15,
18
19     while (!Serial); //
20
21     msd.connect();
22
23     while (!msd.connect
24         Serial.print("MSD
25         delay(1000);
26     }
27
28     Serial.println("Mou

```

After logging data, remove the USB stick from your board, and insert it in your computer to see the data logged:



Data logged in .txt file.

USB Host Keyboard

It is possible to connect generic USB keyboards to the GIGA R1's USB-A connector without any additional circuitry.

The library used for this can be downloaded through Github.

USBHostGiga

Please note that this library is in **Alpha** development stage. This means support is experimental and examples may not function as expected. Future versions of this library may break the example provided below.

The USBHostGiga library is not available in the Arduino IDE and needs to be



installed manually. You can do so by navigating to

Sketch >
Include Library >
Add .ZIP Library.

```
1  #include "USBHostGiga.h"
2
3  //REDIRECT_STDOUT_TO(Serial)
4  Keyboard keyb;
5  HostSerial ser;
6
7  void setup() {
8      // put your setup code here
9      Serial.begin(115200);
10     while (!Serial);
11     pinMode(PA_15, OUTPUT);
12     keyb.begin();
13     ser.begin();
14 }
15
16
17 void loop() {
18     if (keyb.available())
19         auto _key = keyb.read();
20     Serial.println(keyb.read());
21 }
22 while (ser.available())
23     auto _char = ser.read();
24     Serial.write(_char);
25 }
26 //delay(1):
```

USB HID

It is possible to turn your GIGA R1 board into a Human Interface Device (**HID**), aka mouse & keyboard, using the **USBHID** library which is included in the GIGA Board Package.

Among other things, you can:

- Create a custom keyboard, or a keyboard accessory,
- Create sophisticated game controllers,
- Accessories for VR/AR applications.

Keyboard

Important! When using the GIGA as a keyboard, make sure to include some sort of delay. Otherwise, you may end up printing things very fast, which can be an annoyance. If this happens nonetheless, double tap the reset button and upload a blank sketch to reset the board.

To emulate a keyboard, we need to include `PluggableUSBHID.h` and `USBKeyboard.h`, and create an object using the `USBkeyboard` constructor.

```
1 #include "PluggableUSBH
2 #include "USBKeyboard.h
3
4 USBKeyboard Keyboard;
```

To send a single character, we can use the `putc()` method.

```
1 Keyboard.putc(97); //pr
```

See the `DEC` column at [ascii-code.com](https://www.asciicode.com) to understand what number you need to print a specific character.

To print a whole string, use the `printf()` method.

```
1 Keyboard.printf("Hello
```

To use modifiers and function keys, use the `key_code()` method.

```
1 Keyboard.key_code(KEY_F
```

To use media keys, use the `media_control()` method.

```
1 Keyboard.media_control(
```



All modifiers, function and media control keys can be found in [this header file](#).

Mouse

To emulate a mouse, we need to include `PluggableUSBHID.h` and `USBMouse.h`, and create an object using the `USBMouse` constructor.

```
1 #include "PluggableUSBHID.h"
2 #include "USBMouse.h"
3
4 USBMouse Mouse;
```

To move the cursor, we can write the **x** and **y** coordinates directly, using the `move()` method.

```
1 Mouse.move(100,100);
```

For clicking the mouse, use the `click()` method.

```
1 Mouse.click(MOUSE_LEFT)
2 Mouse.click(MOUSE_RIGHT)
3 Mouse.click(MOUSE_MIDDLE)
```

For double clicking the mouse, use the `double_click()` method.

```
1 Mouse.double_click(MOUS
```

To press and release the buttons on the mouse, we can use the `press()` and `release()` methods. This way, we can define how long we want the button to be pressed for.

```
1 Mouse.press(MOUSE_LEFT)
2 Mouse.press(MOUSE_RIGHT)
3 Mouse.press(MOUSE_MIDDL
4
5 delay(1000);
6
7 Mouse.release();
```

Summary

The goal with this guide was to provide a summary of all the GIGA R1's features, including:

Enabling and disabling the USB-A port.

Read & Write to a USB mass storage device (MSD).

Connecting keyboards and reading key presses.

Emulate a mouse/keyboard through the HID interface.

Suggest changes Need support?

The content on docs.arduino.cc is facilitated by the [Help Center](#), [Ask the Arduino Forum](#), [Discover Arduino](#)

License

The Arduino documentation is licensed under the [Creative Commons](#)

[GitHub repository.](#)
If you see anything wrong, you can edit this page [here](#).

[Share Alike 4.0](#) license.

Was this article helpful?

